# Reproducibility

# Reproducibility I

Reproducibility in GNN is a source- and run-binding contract across the 0–24 processing steps. Fixed-source parsing and token production are distinct from stochastic inference, dependency provisioning, and environment-dependent execution. Reproducing a reported outcome requires the selected inputs, configuration, source revision, dependencies, dtype, seed, and retained command and artifact receipts. This section lists only commands that exist in the repository, so that a reader with a clean checkout can reproduce the pipeline, the validation gates, and this manuscript itself.

## Pipeline Smoke Run I

The fastest way to confirm a working installation is to drive the full pipeline over the discrete model family without invoking the optional LLM steps:

```
uv run python src/gnn/main.py \
  --target-dir input/gnn_files/discrete \
  --output-dir /tmp/gnn-smoke \
  --skip-llm
```

## Pipeline Smoke Run I

This parses the discrete GNN files, runs visualization and rendering across the maintained backends, and writes all artifacts under the chosen output directory. The `--skip-llm` flag disables the LLM processing step; it does not establish network isolation or guarantee that every other step executes. Default framework selection excludes the experimental targets, and dependency readiness, model-kind admission, configured steps, and budgets determine the actual work. Inspect the current-run statuses and terminal exit code (0 success, 1 error, 2 warning); missing or unfinished required work must remain a non-success outcome. To exercise every registered family rather than a single one, drive the manifest through the model-family acceptance gate given below: pointing `--target-dir` at `input/gnn_files` covers that tree's 12 corpus directories. All 9 registered family target

directories lie inside that tree, so a single invocation reaches every registered family.

The discrete family exercises the categorical kind end to end. The continuous linear-Gaussian kind smoke-runs the same way, and the contrast between the two runs is itself a check of the per-kind contract:

```
uv run python src/gnn/main.py \
  --target-dir input/gnn_files/continuous \
  --output-dir /tmp/gnn-smoke-continuous \
  --skip-llm
```

## Pipeline Smoke Run I

This selects the continuous specifications for the same pipeline. Admitted, ready continuous-capable lanes can render and execute filtering (and, for the closed-loop exemplar, belief-steering) programs; categorical-only targets record `unsupported`, while readiness and budget failures retain their own outcomes. A reader comparing the two receipts sees the kind taxonomy behaving as described in sec. 3: same pipeline, same steps, per-kind rendering and execution reach.

## Validation Gates I

GNN provides strict gates with distinct verification scopes. The model-family acceptance gate runs the selected manifest families and evaluates their declared acceptance profiles; its receipt must record actual execution, unsupported and skipped work, and failures:

```
uv run python scripts/run_model_family_acceptance.py
\
  --manifest input/model_family_manifest.json \
  --output-dir output/model_family_acceptance
  --strict
```

## Validation Gates I

The semantic-fidelity gate verifies that a parse $\rightarrow$ serialize $\rightarrow$ parse round trip preserves variables, edges, dimensions, parameter shapes and values, parameter metadata, equations, time semantics, and ontology mappings across the 9 model families; the cross-framework gate profiles the 7 maintained backends (PyMDP, RxInfer.jl, JAX, NumPyro, PyTorch, ActiveInference.jl, DisCoPy) — refusing any framework outside that set — and records explicit compatible and unsupported statuses rather than silently degrading. Both write their ledgers to an output directory of your choosing:

## Validation Gates I

```
uv run python scripts/run_semantic_fidelity_gate.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/semantic_fidelity --strict
uv run python
scripts/run_cross_framework_reliability.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/cross_framework --strict
```

## Validation Gates I

Under `--strict`, required mismatches make these gates exit non-zero, so these commands double as assertions in an automated reproduction run. Code-quality reproducibility is enforced separately through the developer command reference: `just lint` runs the Ruff linter over `src` and `scripts`, and the broader `just quality` recipe chains formatting, terminology, documentation, type, and security checks for a full pre-commit gate.

## Manuscript Reproducibility I

This manuscript is itself a reproducible artifact. Every quantitative value in the prose — the pipeline step count, the family and backend counts, the source and test inventories — is a token rather than a hard-coded literal, and the deterministic producer regenerates all of them from the tracked files at the current commit:

```
uv run python
scripts/z_generate_manuscript_variables.py
```

That command recomputes the {{...}} tokens, persists them to output/data/manuscript_variables.json for audit, and hydrates the manuscript sources into output/manuscript/. The manuscript's own figures are rebuilt from the same token map:

```
uv run python -m scripts.manuscript_build_figures
```

The hydrated sources are then rendered to PDF by the docxology template's render stage. That stage lives in a separate checkout of the template, with this repository checked out inside it as a real directory at `projects/active/GeneralizedNotationNotation` (the template's analysis-read custody refuses a symlinked project); run from the template root:

## Manuscript Reproducibility I

```
uv run --frozen python
scripts/pipeline/stage_03_render.py \
  --project GeneralizedNotationNotation
```

The render needs a LaTeX installation providing the packages listed in `manuscript/preamble.md` plus `seqsplit`; the template guards `seqsplit` with `\IfFileExists`, so a missing copy degrades rather than failing the build.

## Manuscript Reproducibility I

Because the variables file is regenerated before rendering, the counts in the rendered PDF track the repository state at the commit recorded in `output/data/manuscript_variables.json` (8e9bcfb48): a code change that alters, for example, the test inventory (660 test files, 6809 test functions) propagates into the prose on the next regeneration without any manual editing.

# Reproducibility Contract I

- Do not cite results that cannot be regenerated or directly traced to a command in this repository.
- Keep generated outputs under `output/` and maintained manuscript source under `manuscript/`. Regenerate owned build artifacts when their inputs change, and preserve the raw command receipts and numerical results needed to substantiate historical runs.
- Express every quantitative claim in the prose as a double-brace `{{...}}` token substituted by `scripts/z_generate_manuscript_variables.py`, never as a hard-coded number.
- Keep private data, credentials, and unpublished sensitive details out of the manuscript and out of version control.
- Record the exact verification commands — the smoke run, the acceptance and fidelity gates, and `just lint` — before marking this manuscript publication-ready.